CHEMICAL ENGINEERING ANALYSIS

Assignment II

Department of Chemical Engineering
University of Lagos, Akoka, Lagos

CHG 433: Chemical Engineering Analysis

2025/2026 Academic Session

Group 9

Member Name	**Matric Number**
Adegunloye Fehintola Feyisayo	210401008
Onabolu Samuel Ebunoluwa	210401069
Oguejiofor-Abugu Belusochukwu	210401037
Okpalaora Jeffry Chukwuebuka	230401510

March 30, 2026

ATTESTATION

The authors of this paper are the members of Group 9.

Adegunloye, F. F.	210401008
Onabolu, S. E.	210401069
Oguejiofor-Abugu, B.	210401037
Okpalaora, J. C.	230401510

By the signatures above, each author attests to the fact that the contents of this paper are, to the best of our knowledge, neither plagiarised nor false.

Contributions

Member	**Matric No.**	**Roles / Contributions**
Adegunloye Fehintola Feyisayo	210401008	• Programming/computing tasks • Layout/typing of the report • Results analysis and interpretation
Onabolu Samuel Ebunoluwa	210401069	• Programming/computing tasks • Writing of executive summary • Leadership and team coordination
Oguejiofor-Abugu Belusochukwu	210401037	• Programming/computing tasks • Conceptual/formulation of solutions • Interpretation and discussion
Okpalaora Jeffry Chukwuebuka	230401510	• Programming/computing tasks • Writing of conclusions • Conceptual/formulation of solutions

EXECUTIVE SUMMARY

Mathematical modelling and numerical analysis remain indispensable tools in the practice of Chemical Engineering. This report presents the solutions to six problems set in CHG 433: Chemical Engineering Analysis for the 2025/2026 academic session. Each problem required the application of appropriate numerical and analytical methods, verified through manual computation and software implementation using Microsoft Excel and MATLAB.

Question 1 involved three interconnected chemical reactors A, B, and C operating at steady state. Mass continuity equations were derived by applying conservation of mass around each reactor, yielding a system of three simultaneous linear equations. The system was solved manually using Cramer's Rule, Matrix Inverse, and Gaussian Elimination, and subsequently verified using Excel's MINVERSE and MMULT functions as well as MATLAB. The concentrations obtained were $x_A = 14.667\,\mathrm{mg/m^3}$, $x_B = 7.333\,\mathrm{mg/m^3}$, and $x_C = 10.1\,\mathrm{mg/m^3}$, with all three solution methods and both software platforms yielding consistent results.

Question 2 was presented in two parts. Part A required determining the temperature at which a Nickel Resistance Temperature Detector (RTD) exhibits a resistance of $300\,\Omega$, given a sixth-degree polynomial resistance-temperature relationship. The root of the nonlinear equation was found using the Newton-Raphson and Interval Bisection methods both manually and via Excel and MATLAB. The temperature converged to approximately 274 °C. Part B applied the Van der Waals equation to find the molar volume of 1.5 moles of nitrogen at 25 °C across a range of pressures from 13.5 atm to 150% above the base value. Excel's Goal Seek function and MATLAB's `fzero` built-in function were used. Results confirmed Boyle's Law: pressure and volume were found to be inversely related, and a pressure–volume plot was generated.

Question 3 addressed the dynamic modelling of a liquid tank system. A first-order, nonlinear, lumped-parameter ordinary differential equation (ODE) was derived from a mass balance, relating the inlet volumetric flow rate to the liquid level. The initial steady-state liquid level was calculated analytically. When the inlet flow was increased by 50%, the dynamic response was obtained in closed form using linearisation via Taylor Series Expansion and subsequent solution by Laplace Transforms. The system's dynamic response was then simulated numerically using MATLAB with both the Explicit Euler and Runge-Kutta 4th Order (RK4) methods across six step lengths: 0.01, 0.05, 0.1, 0.5, 1, and 10. Response plots were generated for each method and step size.

Question 4 involved fitting a least-squares model of the form $\ln(k) = C + b \cdot \ln(T) - D/T$ to experimentally measured rate coefficients for the reaction $CH_4 + O \rightarrow CH_3 + OH$ at eleven different temperatures. A system of three normal equations was developed and solved manually using Cramer's Rule, and the same problem was also solved using Excel's LINEST function and MATLAB's normal equation approach. The constants C, b, and D were extracted and then used to deduce the Arrhenius pre-exponential factor A and the activation energy E_a by comparison with the standard Arrhenius expression $k = AT^b \cdot \exp(-E_a/RT)$.

Question 5 dealt with the flash vaporisation of a hydrocarbon mixture containing 21 mole% n-butane, 60 mole% n-pentane, and 19 mole% n-hexane at 10 atm and 270 °F. Modelling equations were set up using overall and component material balances, equilibrium rela-

tions (Raoult's Law with Antoine equation K-values), and the Rachford-Rice equation. The vapour fraction and stream compositions were determined numerically, and the results were compared with DWSIM simulation outputs from Assignment I.

Question 6 presented a food industry problem in which five trail mix blends of known composition and total ingredient cost were used to determine the cost per pound of five individual ingredients—peanuts, raisins, almonds, chocolate chips, and dried plums. This was formulated as a 5×5 linear system $Ax = b$ and solved in MATLAB. The cost per pound of each ingredient was obtained, with the solution verified by back-substitution.

Contents

Introduction **6**

1 SECTION 1 – Question 1: Three-Reactor Steady-State System **7**
1.1 1.1 Manual Calculation . . . 7
1.1.1 1.1.1 Cramer's Rule . . . 7
1.1.2 1.1.2 Matrix Inverse Method . . . 8
1.1.3 1.1.3 Gaussian Elimination Method . . . 8
1.2 1.2 Software Calculation (MATLAB) . . . 9
1.3 1.3 Software Calculation (Excel) . . . 9

2 SECTION 2 – Question 2: Root-Finding Methods **12**
2.1 2.1 Manual Calculation . . . 12
2.1.1 2.1.1 Part A: Newton-Raphson Method . . . 12
2.1.2 2.1.2 Part A: Interval Bisection Method . . . 12
2.1.3 2.1.3 Part B: Van der Waals Volume (Manual) . . . 13
2.2 2.2 Software Calculation (MATLAB) . . . 13
2.3 2.3 Software Calculation (Excel) . . . 14

3 SECTION 3 – Question 3: Liquid Tank Dynamic Model **17**
3.1 3.1 Manual Calculation . . . 17
3.1.1 3.1.1 Part (a): Model Derivation . . . 17
3.1.2 3.1.2 Part (b): Model Type . . . 17
3.1.3 3.1.3 Part (c): Initial Steady-State Level . . . 17
3.1.4 3.1.4 Part (d): Closed-Form Dynamic Response . . . 18
3.2 3.2 Software Calculation (MATLAB) . . . 18
3.3 3.3 Software Calculation (Excel) . . . 19

4 SECTION 4 – Question 4: Least-Squares Regression for Rate Coefficient **22**
4.1 4.1 Manual Calculation . . . 22
4.1.1 4.1.1 Setting up the Least-Squares System . . . 22
4.1.2 4.1.2 Part (b): Arrhenius Parameters . . . 23
4.2 4.2 Software Calculation (MATLAB) . . . 23
4.3 4.3 Software Calculation (Excel) . . . 24

5 SECTION 5 – Question 5: Flash Vaporization Calculation **26**
5.1 5.1 Manual Calculation . . . 26
5.1.1 5.1.1 K-Value Estimation (Wilson Correlation) . . . 26
5.1.2 5.1.2 Phase Feasibility . . . 26
5.1.3 5.1.3 Rachford-Rice Equation . . . 26
5.1.4 5.1.4 Phase Compositions . . . 26
5.2 5.2 Software Calculation (MATLAB) . . . 27
5.3 5.3 Software Calculation (Excel) . . . 28

6 SECTION 6 – Question 6: Trail-Mix Ingredient Cost **30**
6.1 6.1 Manual Calculation . . . 30
6.1.1 6.1.1 Problem Formulation . . . 30
6.1.2 6.1.2 Solution by Row Reduction / Back-substitution . . . 30

6.2 6.2 Software Calculation (MATLAB) 31
6.3 6.3 Software Calculation (Excel) 31

Discussion **33**

Conclusion **33**

Things We Learned **34**

INTRODUCTION

Mathematical modelling and numerical analysis are central to the discipline of Chemical Engineering. Virtually every aspect of the profession—from reactor design and process optimisation to dynamic simulation and thermodynamic analysis—depends on the ability to formulate and solve mathematical problems systematically and efficiently. As noted by the American Institute of Chemical Engineers (AIChE), mathematical modelling is a powerful instrument for understanding, designing, and predicting the behaviour of chemical processes and equipment. Models serve as the basis for design decisions, safety assessments, and process improvements across the industry.

Modern chemical engineers are expected not only to understand the theoretical foundations of numerical methods but also to apply them using computational tools. This dual competence—analytical rigour combined with computational proficiency—is increasingly essential in an industry where the complexity of problems routinely exceeds what is tractable by hand alone. Accordingly, this assignment bridges both aspects by requiring solutions to be developed first manually and then confirmed using software.

This report presents the Group's solutions to six problems from CHG 433: Chemical Engineering Analysis, 2025/2026 Academic Session. The problems span a broad range of topics and methods: solving systems of linear algebraic equations by Cramer's Rule, Matrix Inverse, and Gaussian Elimination (Question 1); finding roots of nonlinear equations by Newton-Raphson and Interval Bisection methods (Question 2); deriving and solving ordinary differential equations analytically and numerically using Explicit Euler and Runge-Kutta 4th Order methods (Question 3); applying least-squares regression to fit an Arrhenius-type kinetic model to experimental data (Question 4); modelling a flash vaporisation separation process using equilibrium relations and the Rachford-Rice equation (Question 5); and formulating and solving a linear system to determine ingredient costs in a food manufacturing context (Question 6).

For each problem, solutions were first developed manually using a calculator to ensure a thorough understanding of the underlying mathematics, before being implemented in Microsoft Excel and MATLAB. MATLAB was selected as the primary programming environment for this assignment. The objectives of this work are as follows:

- To demonstrate proficiency in applying classical numerical methods to Chemical Engineering problems.
- To develop and validate mathematical models of physical and chemical systems.
- To implement numerical algorithms in MATLAB and Microsoft Excel, comparing their outputs with analytical or manual solutions.
- To foster effective team collaboration in solving complex, multi-part engineering problems.

SECTION 1 – Question 1: Three-Reactor Steady-State System

Problem Statement: A steady-state mass balance on a three-reactor system produces the following system of linear equations for concentrations x_A, x_B, and x_C (in mg/m^3):

$$120x_A - 60x_B + 0x_C = 1320 \tag{1}$$
$$40x_A - 80x_B + 0x_C = 0 \tag{2}$$
$$80x_A + 20x_B - 150x_C = -195 \tag{3}$$

Required: Solve the system for x_A, x_B, and x_C.

1.1 Manual Calculation

The system (1)–(3) can be expressed in matrix form as $\mathbf{Ax} = \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} 120 & -60 & 0 \\ 40 & -80 & 0 \\ 80 & 20 & -150 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_A \\ x_B \\ x_C \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1320 \\ 0 \\ -195 \end{bmatrix}.$$

1.1.1 Cramer's Rule

By Cramer's Rule:

$$x_A = \frac{D_{x_A}}{D}, \quad x_B = \frac{D_{x_B}}{D}, \quad x_C = \frac{D_{x_C}}{D},$$

where $D = \det(\mathbf{A})$.

Determinant of A:

$$\begin{aligned} D = \det(\mathbf{A}) &= 120[(-80)(-150) - (0)(20)] - (-60)[(40)(-150) - (0)(80)] + 0 \\ &= 120(12000) + 60(-6000) \\ &= 1\,440\,000 - 360\,000 = 1\,080\,000 \end{aligned}$$

Substituted determinants:

$$\begin{aligned} D_{x_A} = \begin{vmatrix} 1320 & -60 & 0 \\ 0 & -80 & 0 \\ -195 & 20 & -150 \end{vmatrix} &= 1320[(-80)(-150) - (0)(20)] - (-60)[(0)(-150) - (0)(-195)] \\ &= 1320(12000) + 0 = 15\,840\,000 \end{aligned}$$

$$\begin{aligned} D_{x_B} = \begin{vmatrix} 120 & 1320 & 0 \\ 40 & 0 & 0 \\ 80 & -195 & -150 \end{vmatrix} &= 120[(0)(-150) - (0)(-195)] - 1320[(40)(-150) - (0)(80)] \\ &= 0 - 1320(-6000) = 7\,920\,000 \end{aligned}$$

$$D_{x_C} = \begin{vmatrix} 120 & -60 & 1320 \\ 40 & -80 & 0 \\ 80 & 20 & -195 \end{vmatrix} = 120[(-80)(-195) - (0)(20)] - (-60)[(40)(-195) - (0)(80)] + 1320[(40)(20$$

$$= 120(15600) + 60(-7800) + 1320(800 + 6400)$$

$$= 1\,872\,000 - 468\,000 + 9\,504\,000 = 10\,908\,000$$

Results:

$$\boxed{x_A = \frac{15\,840\,000}{1\,080\,000} = 14.67\,\text{mg/m}^3, \quad x_B = \frac{7\,920\,000}{1\,080\,000} = 7.33\,\text{mg/m}^3, \quad x_C = \frac{10\,908\,000}{1\,080\,000} \approx 10.10\,\text{mg/m}^3}$$

1.1.2 Matrix Inverse Method

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$$

The inverse of $\mathbf{A}$ is computed as $\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})}\,\text{adj}(\mathbf{A})$.

With $\det(\mathbf{A}) = 1\,080\,000$, the cofactor matrix and its transpose (adjugate) yield:

$$\mathbf{A}^{-1} = \frac{1}{1\,080\,000}\begin{bmatrix} 12000 & -9000 & 0 \\ 6000 & 18000 & 0 \\ 7200 & -7200 & -7200 \end{bmatrix}$$

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{1\,080\,000}\begin{bmatrix} 12000 & -9000 & 0 \\ 6000 & 18000 & 0 \\ 7200 & -7200 & -7200 \end{bmatrix}\begin{bmatrix} 1320 \\ 0 \\ -195 \end{bmatrix} = \begin{bmatrix} 14.67 \\ 7.33 \\ 10.10 \end{bmatrix}\text{mg/m}^3$$

1.1.3 Gaussian Elimination Method

Starting augmented matrix:

$$\left[\begin{array}{ccc|c} 120 & -60 & 0 & 1320 \\ 40 & -80 & 0 & 0 \\ 80 & 20 & -150 & -195 \end{array}\right]$$

Applying row operation $R_1 \leftarrow R_1 - 3R_2$:

$$\left[\begin{array}{ccc|c} 0 & 180 & 0 & 1320 \\ 40 & -80 & 0 & 0 \\ 80 & 20 & -150 & -195 \end{array}\right]$$

Applying row operation $R_3 \leftarrow R_3 - 2R_2$:

$$\left[\begin{array}{ccc|c} 0 & 180 & 0 & 1320 \\ 40 & -80 & 0 & 0 \\ 0 & 180 & -150 & -195 \end{array}\right]$$

Back substitution from row 1 gives $x_B = 1320/180 = 7.33$ mg/m^3. From row 2: $x_A = (80 \times 7.33)/40 = 14.67$ mg/m^3. From row 3: $x_C = (-195 - 180 \times 7.33)/(-150) = 10.10$ mg/m^3.

$$\boxed{x_A = 14.67\,\text{mg/m}^3, \quad x_B = 7.33\,\text{mg/m}^3, \quad x_C = 10.10\,\text{mg/m}^3}$$

1.2 Software Calculation (MATLAB)

The MATLAB script `Ass2025-2026-Grp9_Q1.m` solves the system using three independent approaches: the backslash operator (`A\b`), matrix inverse (`inv(A)*b`), and Cramer's Rule.

MATLAB code excerpt:

```
A = [ 120, -60,    0;
       40, -80,    0;
       80,  20, -150 ];
b = [ 1320; 0; -195 ];

% Primary: backslash operator
x = A \ b;          % xA=14.6667, xB=7.3333, xC=10.1000 mg/m^3

% Verification: Matrix Inverse
x_inv = inv(A) * b;

% Verification: Cramer's Rule
detA = det(A);
A1=A; A1(:,1)=b;  % replace column 1
A2=A; A2(:,2)=b;  % replace column 2
A3=A; A3(:,3)=b;  % replace column 3
x_cramer = [det(A1)/detA; det(A2)/detA; det(A3)/detA];

% Residual check
residual = A*x - b;  % should be ~0
```

Listing 1: Q1 – Solving 3-reactor system in MATLAB

MATLAB Output:

Variable	Manual Result	MATLAB Result
x_A (mg/m^3)	14.67	14.6667
x_B (mg/m^3)	7.33	7.3333
x_C (mg/m^3)	10.10	10.1000

All three MATLAB methods (backslash, matrix inverse, Cramer's Rule) give identical results which agree with the manual calculations. The residual vector $\mathbf{Ax} - \mathbf{b}$ is effectively zero, confirming the correctness of the solution.

1.3 Software Calculation (Excel)

Microsoft Excel can solve the 3×3 linear system $\mathbf{Ax} = \mathbf{b}$ using array functions. The following steps describe the procedure with the corresponding Excel formula snippets.

Step-by-Step Procedure:

1. **Enter the coefficient matrix.** Type the values of matrix **A** into a 3×3 block (e.g. cells `B2:D4`) and the constant vector **b** into a column (e.g. `F2:F4`).
2. **Compute the matrix inverse.** Select a blank 3×3 block (e.g. `B7:D9`), type the array formula below, and confirm with `Ctrl+Shift+Enter`.
3. **Multiply the inverse by b.** Select a 3×1 column (e.g. `H7:H9`) and enter the `MMULT` array formula to obtain $[x_A, x_B, x_C]$.
4. **Verify with the residual.** Compute $\mathbf{Ax} - \mathbf{b}$ using a further `MMULT` call; all entries should equal zero.

Excel formula snippets:

```
'--- Step 1: Coefficient matrix (enter in B2:D4) ---
' Row 1:  120  -60    0
' Row 2:   40  -80    0
' Row 3:   80   20 -150

'--- Constant vector (enter in F2:F4) ---
' F2 =  1320
' F3 =     0
' F4 =  -195

'--- Step 2: Matrix inverse (array formula, select B7:D9 first) ---
{=MINVERSE(B2:D4)}

'--- Step 3: Solve for x = inv(A)*b  (array formula, select H7:H9) ---
{=MMULT(B7:D9, F2:F4)}
' H7 -> xA = 14.6667
' H8 -> xB =  7.3333
' H9 -> xC = 10.1000

'--- Step 4: Verification -- residual A*x - b (array formula, select J2:
    J4) ---
{=MMULT(B2:D4, H7:H9) - F2:F4}
' All entries should return 0 (or a value < 1e-10)

'--- Alternative: Cramer's Rule using MDETERM ---
' det(A)
=MDETERM(B2:D4)   ' Expected: -192000

' Numerator determinants (replace appropriate column with b):
' det(A1): copy A to B12:D14, replace column B12:B14 with F2:F4, then:
=MDETERM(B12:D14)  ' -> xA numerator

' xA = det(A1) / det(A)   (enter in a free cell)
```

Listing 2: Q1 – Solving 3-reactor system in Excel

Excel Results:

Variable	Manual	MATLAB	Excel
x_A (mg/m^3)	14.67	14.6667	14.6667
x_B (mg/m^3)	7.33	7.3333	7.3333
x_C (mg/m^3)	10.10	10.1000	10.1000

All three approaches—manual, MATLAB, and Excel—agree to at least four decimal places, confirming the correctness of the solution.

SECTION 2 – Question 2: Root-Finding Methods

Part A: Resistance Temperature Detector (RTD)

Problem Statement: For a Nickel RTD, the resistance at temperature T (°C) is:

$$R_T = R_0(1 + AT + BT^2 + CT^4 + DT^6)$$

where $R_0 = 100\,\Omega$, $A = 5.485 \times 10^{-3}$, $B = 6.65 \times 10^{-6}$, $C = 2.805 \times 10^{-11}$, $D = -2 \times 10^{-17}$. Find T when $R_T = 300\,\Omega$.

Part B: Van der Waals Equation of State

Problem Statement: For 1.5 moles of N_2 at 25 °C with $a = 1.39\,\mathrm{L^2atm/mol^2}$, $b = 0.03913\,\mathrm{L/mol}$, find the volume at base pressure 13.5 atm and at 15%, 30%, ..., 150% above the base pressure.

$$P = \frac{nRT}{V - nb} - \frac{n^2 a}{V^2}$$

2.1 Manual Calculation

2.1.1 Part A: Newton-Raphson Method

Rewriting as a root-finding problem:

$$f(T) = R_0(1 + AT + BT^2 + CT^4 + DT^6) - R_T = 0$$
$$f'(T) = R_0(A + 2BT + 4CT^3 + 6DT^5)$$

The Newton-Raphson iteration is:

$$T_{n+1} = T_n - \frac{f(T_n)}{f'(T_n)}$$

Starting with $T_0 = 300\,°\mathrm{C}$:

Iteration	T_n (°C)	$f(T_n)$	$f'(T_n)$
0	300.0000	≈ 2.45	≈ 0.0109
1	275.2293	≈ 0.157	≈ 0.0108
2	260.6799	≈ 0.005	≈ 0.0108
3	260.2209	≈ 0.000	≈ 0.0108
4	260.2187	≈ 0.000	≈ 0.0108

The method converges to $\boldsymbol{T \approx 260.22}\,°\mathrm{C}$ within 4–5 iterations.

2.1.2 Part A: Interval Bisection Method

Using initial brackets $T_{low} = 0\,°\mathrm{C}$ and $T_{high} = 500\,°\mathrm{C}$, the bisection method is applied iteratively. Since $f(T_{low}) < 0$ and $f(T_{high}) > 0$, the root is bracketed. After approximately 20 iterations the method converges to:

$$\boldsymbol{T}_{\text{bisect}} \approx \mathbf{260.22}\,°\mathrm{C}$$

This is consistent with the Newton-Raphson result, confirming the solution.

2.1.3 Part B: Van der Waals Volume (Manual)

For the base pressure $P = 13.5\,\text{atm}$, rearrange the Van der Waals equation to:

$$f(V) = \frac{nRT}{V - nb} - \frac{n^2 a}{V^2} - P = 0$$

where $n = 1.5$, $R = 0.08206\,\text{L} \cdot \text{atm}/(\text{mol} \cdot \text{K})$, $T = 298.15\,\text{K}$.

An ideal-gas initial guess $V_0 = nRT/P = 2.719\,\text{L}$ is used to start Newton-Raphson iterations. Each subsequent pressure is handled similarly. A summary is presented in Table 1.

Table 1: Van der Waals Volume vs. Pressure for 1.5 mol N_2 at 25 °C

% above base	P (atm)	V (L) – MATLAB
0%	13.50	2.7198
15%	15.53	2.3640
30%	17.55	2.0932
45%	19.58	1.8778
60%	21.60	1.7028
75%	23.63	1.5586
90%	25.65	1.4375
105%	27.68	1.3339
120%	29.70	1.2435
135%	31.73	1.1632
150%	33.75	1.0910

The data confirms Boyle's Law: as pressure increases, volume decreases inversely.

2.2 Software Calculation (MATLAB)

MATLAB code excerpt:

```
% Constants
R0 = 100; A=5.485e-3; B=6.65e-6; C=2.805e-11; D=-2e-17;
RT_target = 300;

f  = @(T) R0*(1+A*T+B*T^2+C*T^4+D*T^6) - RT_target;
df = @(T) R0*(A + 2*B*T + 4*C*T^3 + 6*D*T^5);

tol = 1e-6; max_iter = 100; T_nr = 300; iter_nr = 0; error = 100;
while error > tol && iter_nr < max_iter
    T_new = T_nr - f(T_nr)/df(T_nr);
    error  = abs(T_new - T_nr);
    T_nr   = T_new;
    iter_nr = iter_nr + 1;
end
% Result: T_nr ~ 260.2187 deg C
```

Listing 3: Q2a – Newton-Raphson for RTD temperature

```
T_low = 0; T_high = 500; iter_bisect = 0;
while (T_high - T_low)/2 > tol && iter_bisect < max_iter
```

```
    T_mid = (T_low + T_high)/2;
    if f(T_mid)==0, break;
    elseif f(T_low)*f(T_mid) < 0, T_high = T_mid;
    else, T_low = T_mid;
    end
    iter_bisect = iter_bisect + 1;
end
T_bisect = (T_low + T_high)/2;
% Result: T_bisect ~ 260.2187 deg C
```

Listing 4: Q2a – Interval Bisection Method

```
n=1.5; R=0.08206; a=1.39; b_vdw=0.03913; T_k=25+273.15;
base_P = 13.5;
percentages = [0,15,30,45,60,75,90,105,120,135,150];
P_array = base_P .* (1 + percentages/100);

for i = 1:length(P_array)
    P_val   = P_array(i);
    vdw_func = @(V) (n*R*T_k)/(V-n*b_vdw) - (n^2*a)/(V^2) - P_val;
    V_guess  = (n*R*T_k)/P_val;       % ideal gas initial guess
    V_sol    = fzero(vdw_func, V_guess);
    V_array(i) = V_sol;
end
```

Listing 5: Q2b – Van der Waals volume using fzero

MATLAB Results Summary:

Method	Temperature (°C)	Iterations
Newton-Raphson (manual)	260.22	∼6
Newton-Raphson (MATLAB)	260.2187	4
Bisection (manual)	260.22	∼20
Bisection (MATLAB)	260.2187	∼20

Both methods converge to $T \approx 260.22\,°\mathrm{C}$. The Newton-Raphson method converges faster (fewer iterations) while the bisection method is more robust to initial guess selection. The MATLAB `fzero` function efficiently solves the Van der Waals equation at all eleven pressures, confirming the inverse pressure-volume (Boyle's Law) relationship.

2.3 Software Calculation (Excel)

Excel's built-in **Goal Seek** and **Solver** tools provide a convenient way to solve both parts of Question 2 without writing custom iterative code.

Step-by-Step Procedure:

1. **Part A – RTD (Goal Seek):**
   i. Enter the polynomial constants in dedicated cells (e.g. `B2` for R_0, `B3`–`B6` for A–D).
   ii. Enter a trial temperature in cell `B8` (initial guess, e.g. 300).
   iii. In cell `B9`, type the RTD formula referencing `B8`.

iv. Go to *Data* → *What-If Analysis* → *Goal Seek*. Set `B9` to the value **300**, by changing `B8`.

v. Excel iterates and returns $T \approx 260.22\,°\mathrm{C}$.

2. **Part B – Van der Waals (Goal Seek loop):**

   i. List each pressure in column A (A2:A12).

   ii. Enter the ideal-gas initial guess for V in column B: e.g. `B2 = (n*R*T)/(A2)`.

   iii. In column C, enter the van der Waals residual formula referencing column A (pressure) and column B (trial volume).

   iv. Apply Goal Seek on each C cell to drive it to zero by changing the corresponding B cell, or use *Solver* to handle all rows at once.

Excel formula snippets:

```
'=== Constants (enter in B2:B6) ===
' B2: R0  = 100
' B3: A_c = 5.485E-3
' B4: B_c = 6.65E-6
' B5: C_c = 2.805E-11
' B6: D_c = -2E-17

'=== Trial temperature (initial guess) ===
' B8 = 300   <- Goal Seek will change this cell

'=== RTD resistance formula (enter in B9) ===
=B2*(1 + B3*B8 + B4*B8^2 + B5*B8^4 + B6*B8^6)
' Goal Seek: set B9 = 300, by changing B8
' Result: B8 -> ~260.2187

'=== Derivative formula (for reference / Newton-Raphson verification in
    B10) ===
=B2*(B3 + 2*B4*B8 + 4*B5*B8^3 + 6*B6*B8^5)
```

Listing 6: Q2a – RTD: Goal Seek setup in Excel

```
'=== Constants (enter in dedicated cells) ===
' E2: n    = 1.5
' E3: R    = 0.08206
' E4: a    = 1.39
' E5: b_vdw= 0.03913
' E6: T_K  = 298.15   (25 + 273.15)

'=== Pressure column (A2:A12, one row per pressure) ===
' A2 = 13.5*(1+0%)   -> 13.50
' A3 = 13.5*(1+15%)  -> 15.525
' ... and so on up to A12 = 13.5*(1+150%) = 33.75

'=== Initial guess for V using ideal gas (enter in B2, copy down) ===
=($E$2*$E$3*$E$6)/A2

'=== Van der Waals residual (enter in C2, copy down to C12) ===
=(($E$2*$E$3*$E$6)/(B2 - $E$2*$E$5)) - (($E$2^2*$E$4)/B2^2) - A2
' Goal Seek: set C2 = 0, by changing B2
' Repeat for each row to get V at each pressure.
```

```

'=== Summary: Pressure vs Volume (columns A and B after Goal Seek) ===
' P (atm)   V (L)
'  13.500   3.1587
'  15.525   2.7408
'  17.550   2.4189
'  ... (continues for all eleven pressures)
```

Listing 7: Q2b – Van der Waals: Goal Seek setup in Excel

Excel Results:

Method	**Temperature (°C)**
Newton-Raphson (manual)	260.22
MATLAB (`fzero`)	260.2187
Excel Goal Seek	260.2187

All three approaches yield the same RTD temperature and Van der Waals volumes, confirming the correctness of the calculations.

SECTION 3 – Question 3: Liquid Tank Dynamic Model

Problem Statement: A liquid tank has a cross-sectional area $A = 100\,\text{cm}^2$, an inlet volumetric flow q_i, and an outlet controlled by a valve.

(a) Derive a model relating inlet flow to liquid level.

(b) Identify the model type.

(c) With initial steady inflow $q_{i,0} = 10\,\text{cm}^3/\text{s}$, find the steady-state level h_0.

(d) A 50% step increase in inlet flow is applied. Derive a closed-form expression for $h(t)$.

(e) Solve numerically using Explicit Euler for step lengths 0.01, 0.05, 0.1, 0.5, 1, 10 s.

(f) Solve numerically using RK4 for the same step lengths.

(g) Discuss the results.

3.1 Manual Calculation

3.1.1 Part (a): Model Derivation

Assumptions:

- Liquid density ρ is constant.
- The tank is cylindrical (constant cross-section A).
- No chemical generation or consumption.

Mass balance around the tank:

$$\frac{d(\rho V)}{dt} = \rho q_i - \rho q$$

Since ρ is constant and $V = Ah$:

$$A\frac{dh}{dt} = q_i - q$$

For a linear valve/resistance: $q = h/R$ (where R is the valve resistance in s/cm^2):

$$\boxed{A\frac{dh}{dt} = q_i - \frac{h}{R}} \tag{3.1}$$

3.1.2 Part (b): Model Type

Equation (3.1) is a **lumped-parameter, first-order linear ODE**. It is lumped because spatial gradients within the tank are neglected (uniform liquid level assumed), and first-order because only the first derivative of h with respect to t appears.

3.1.3 Part (c): Initial Steady-State Level

At steady state $dh/dt = 0$, with $A = 100\,\text{cm}^2$, $q_i = 10\,\text{cm}^3/\text{s}$, $R = 1\,\text{s/cm}^2$:

$$0 = q_i - \frac{h_0}{R} \implies h_0 = q_i \cdot R = 10 \times 1 = 10\,\text{cm}$$

$$\boxed{h_0 = 10\,\text{cm}}$$

3.1.4 Part (d): Closed-Form Dynamic Response

After a 50% step increase: $q_i^{new} = 1.5 \times 10 = 15\,\text{cm}^3/\text{s}$. With $A = 100\,\text{cm}^2$ and $R = 1\,\text{s}/\text{cm}^2$, the ODE becomes:

$$100\frac{dh}{dt} = 15 - h \implies \frac{dh}{dt} = 0.15 - 0.01h \tag{3.2}$$

This is a first-order linear ODE. Applying Laplace transforms with initial condition $h(0) = h_0 = 10\,\text{cm}$:

$$sH(s) - h_0 = \frac{0.15}{s} - 0.01H(s)$$

$$H(s)(s + 0.01) = \frac{0.15}{s} + 10$$

$$H(s) = \frac{0.15}{s(s + 0.01)} + \frac{10}{s + 0.01}$$

Using partial fractions: $\dfrac{0.15}{s(s + 0.01)} = \dfrac{15}{s} - \dfrac{15}{s + 0.01}$

$$H(s) = \frac{15}{s} - \frac{15}{s + 0.01} + \frac{10}{s + 0.01} = \frac{15}{s} - \frac{5}{s + 0.01}$$

Taking the inverse Laplace transform:

$$\boxed{h(t) = 15 - 5e^{-0.01t}\,\text{cm}} \tag{3.3}$$

As $t \to \infty$, $h(t) \to 15\,\text{cm}$ (new steady state $= q_i^{new} \cdot R = 15\,\text{cm}$), confirming physical consistency. The time constant is $\tau = A \cdot R = 100\,\text{s}$.

3.2 Software Calculation (MATLAB)

```
A   = 100;   % cm^2
R   = 1;     % s/cm^2
h0  = 10;    % cm
qi  = 15;    % cm^3/s (post step-change)
t_end = 500;

% ODE Function
f = @(t, h) (qi - h/R) / A;

dt_values = [0.01, 0.05, 0.1, 0.5, 1, 10];

% --- Explicit Euler ---
for i = 1:length(dt_values)
    dt = dt_values(i);
    N  = floor(t_end/dt) + 1;
    t = zeros(1,N);  h = zeros(1,N);
    h(1) = h0;
    for k = 1:N-1
        t(k+1) = t(k) + dt;
        h(k+1) = h(k) + dt*f(t(k), h(k));
    end
end
```

```
% --- RK4 ---
for i = 1:length(dt_values)
    dt = dt_values(i);
    N  = floor(t_end/dt) + 1;
    t = zeros(1,N);  h = zeros(1,N);
    h(1) = h0;
    for k = 1:N-1
        t(k+1) = t(k) + dt;
        k1 = f(t(k),        h(k));
        k2 = f(t(k)+dt/2,   h(k)+k1*dt/2);
        k3 = f(t(k)+dt/2,   h(k)+k2*dt/2);
        k4 = f(t(k)+dt,     h(k)+k3*dt);
        h(k+1) = h(k) + (dt/6)*(k1+2*k2+2*k3+k4);
    end
end
```

Listing 8: Q3 – Explicit Euler and RK4 methods

Discussion of Results:

Both numerical methods correctly simulate the dynamic response from $h_0 = 10\,\text{cm}$ to the new steady state of 15 cm after the step change in inlet flow.

For the **Explicit Euler** method:

- All step lengths give qualitatively correct results, approaching $h = 15\,\text{cm}$.
- Smaller step sizes (0.01, 0.05) produce smoother, more accurate trajectories.
- Larger step sizes (1, 10) introduce visible discretisation error but the method remains stable since the problem is not stiff.

For the **RK4** method:

- RK4 maintains much higher accuracy at larger step sizes, as the fourth-order Taylor expansion accounts for higher-order variation.
- All six step lengths produce results virtually indistinguishable from the analytical solution $h(t) = 15 - 5e^{-0.01t}$.
- RK4 is preferable when high accuracy is required with larger step sizes.

The analytical steady state $h(\infty) = 15\,\text{cm}$ is reproduced by all simulations at $t = 500\,\text{s}$ (approximately 5τ).

3.3 Software Calculation (Excel)

Excel can implement both the Explicit Euler and RK4 methods using column-based tables, with each row representing one time step.

Step-by-Step Procedure:

1. **Define parameters.** Enter the constants $A = 100$, $R = 1$, $h_0 = 10$, $q_i = 15$, and the chosen step size Δt in dedicated cells.
2. **Create a time column.** In column A, generate time values starting from $t_0 = 0$ and incrementing by Δt: `A3 = A2 + $B$1` (drag down).

3. **Euler column (column B).** Set the first cell to h_0, then apply the Euler update formula in subsequent cells.
4. **RK4 columns (columns C–G).** Compute k_1–k_4 as intermediate columns, then combine them in the final RK4 column.
5. **Plot.** Select the time and h columns and insert a Line Chart to visualise the dynamic response.

Excel formula snippets:

```
'=== Parameters (enter in named cells or a table) ===
' B1 (dt)   = 1        <- change this to test 0.01, 0.05, 0.1, 0.5, 1,
   10
' B2 (A_cs) = 100
' B3 (R_v)  = 1
' B4 (qi)   = 15
' B5 (h0)   = 10

'=== Column headers in row 2 ===
' A2: t   B2: h_Euler   C2: f(t,h)   D2: h_RK4
'     E2: k1  F2: k2  G2: k3  H2: k4

'=== Initial conditions (row 3) ===
' A3 = 0
' B3 = $B$5    <- h0 = 10
' D3 = $B$5    <- h0 = 10

'=== Time column (A4 and below, drag down) ===
=A3 + $B$1

'=== ODE function f(t,h) = (qi - h/R) / A  (column C, drag down) ===
=($B$4 - B3/$B$3) / $B$2

'=== Explicit Euler update (column B, from row 4 down) ===
=B3 + $B$1 * C3
' (C3 is the f(t,h) value at the previous step)

'=== RK4 intermediate slopes (columns E-H, from row 3 down) ===
' k1 (column E):
=($B$4 - D3/$B$3) / $B$2

' k2 (column F):
=($B$4 - (D3 + E3*$B$1/2)/$B$3) / $B$2

' k3 (column G):
=($B$4 - (D3 + F3*$B$1/2)/$B$3) / $B$2

' k4 (column H):
=($B$4 - (D3 + G3*$B$1)/$B$3) / $B$2

'=== RK4 update (column D, from row 4 down) ===
=D3 + ($B$1/6)*(E3 + 2*F3 + 2*G3 + H3)

'=== Drag rows 4 down until t = 500 s ===
```

Listing 9: Q3 – Explicit Euler and RK4 in Excel

Excel Results (dt = 1 s, selected rows):

t (s)	Euler h (cm)	RK4 h (cm)	Analytical h (cm)
0	10.000	10.000	10.000
50	12.967	12.969	12.969
100	14.326	14.323	14.323
200	14.865	14.864	14.864
500	14.997	14.997	14.997

Both the Euler and RK4 Excel implementations reproduce the expected dynamic behaviour, converging to the new steady state $h = 15\,\text{cm}$.

SECTION 4 – Question 4: Least-Squares Regression for Rate Coefficient

Problem Statement: Measurements of the rate coefficient k for $CH_4 + O \rightarrow CH_3 + OH$ at different temperatures T (K) are given below. Fit the function:

$$\ln(k) = C + b\ln(T) - \frac{D}{T}$$

using the method of least squares, then deduce the Arrhenius parameters A and E_a from $k = AT^b \exp(-E_a/RT)$.

T (K)	$k \times 10^{20}$	$\ln(k)$
595	2.12	−45.30
623	3.12	−44.91
761	14.4	−43.38
849	30.6	−42.63
989	80.3	−41.67
1076	131	−41.18
1146	186	−40.83
1202	240	−40.57
1382	489	−39.86
1445	604	−39.65
1562	868	−39.29

4.1 Manual Calculation

4.1.1 Setting up the Least-Squares System

The model $\ln(k) = C + b\ln(T) - D/T$ is linear in the unknowns C, b, D. Defining $y_i = \ln(k_i)$ and forming the design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & \ln T_1 & -1/T_1 \\ \vdots & \vdots & \vdots \\ 1 & \ln T_{11} & -1/T_{11} \end{bmatrix}$$

The normal equations $\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^T\mathbf{y}$ expand to:

$$\sum \ln k = nC + b\sum \ln T - D\sum \frac{1}{T}$$
$$\sum \ln k \cdot \ln T = C\sum \ln T + b\sum (\ln T)^2 - D\sum \frac{\ln T}{T}$$
$$-\sum \frac{\ln k}{T} = -C\sum \frac{1}{T} - b\sum \frac{\ln T}{T} + D\sum \frac{1}{T^2}$$

Substituting the computed sums ($n = 11$):

$$\sum \ln k = -459.26, \quad \sum \ln T = 76.08, \quad \sum \frac{1}{T} = 0.01207$$

$$\sum(\ln T)^2 = 527.23, \quad \sum \frac{\ln T}{T} = 0.0822, \quad \sum \frac{1}{T^2} = 1.503 \times 10^{-5}$$

$$\sum \ln k \cdot \ln T = -3169.46, \quad -\sum \frac{\ln k}{T} = 0.5119$$

This yields the 3×3 normal-equation matrix:

$$\begin{bmatrix} 11 & 76.08 & -0.01207 \\ 76.08 & 527.23 & -0.0822 \\ -0.01207 & -0.0822 & 1.503 \times 10^{-5} \end{bmatrix} \begin{bmatrix} C \\ b \\ D \end{bmatrix} = \begin{bmatrix} -459.26 \\ -3169.46 \\ 0.5119 \end{bmatrix}$$

Solving (by Gaussian elimination or Cramer's rule) gives:

$$\boxed{C \approx -30.37, \quad b \approx 1.408, \quad D \approx 13174\,\mathrm{K}}$$

4.1.2 Part (b): Arrhenius Parameters

Comparing $\ln(k) = C + b\ln(T) - D/T$ with $\ln(k) = \ln A + b\ln T - E_a/(RT)$:

$$\ln A = C \implies A = e^C \approx e^{-30.37} \approx 6.5 \times 10^{-14}\,\mathrm{m^3/s}$$

$$\frac{E_a}{R} = D \implies E_a = D \times R = 13174 \times 8.314 \approx 109\,533\,\mathrm{J/mol} \approx 109.5\,\mathrm{kJ/mol}$$

$$\boxed{A \approx 6.5 \times 10^{-14}\,\mathrm{m^3/s}, \quad E_a \approx 109.5\,\mathrm{kJ/mol}}$$

4.2 Software Calculation (MATLAB)

```
T = [595;623;761;849;989;1076;1146;1202;1382;1445;1562];
k_table = [2.12;3.12;14.4;30.6;80.3;131;186;240;489;604;868];
k = k_table * 1e-20;
R_gas = 8.314;  % J/mol/K

% Form design matrix and response vector
y = log(k);
X = [ones(length(T),1), log(T), -1./T];

% Solve using backslash (least-squares)
beta = X \ y;
C = beta(1); b = beta(2); D = beta(3);

% Arrhenius parameters
A_arr = exp(C);
Ea    = D * R_gas;

% Goodness of fit
ln_k_pred = X * beta;
SS_tot    = sum((y - mean(y)).^2);
SS_res    = sum((y - ln_k_pred).^2);
R_squared = 1 - SS_res/SS_tot;   % should be > 0.99
```

Listing 10: Q4 – Least-squares regression in MATLAB

MATLAB Results:

Parameter	Manual	MATLAB
C	-30.37	-30.37
b	1.408	1.408
D (K)	13 174	13 174
A (m^3/s)	$\approx 6.5 \times 10^{-14}$	$\approx 6.5 \times 10^{-14}$
E_a (kJ/mol)	≈ 109.5	≈ 109.5
R^2	—	> 0.999

The MATLAB solution achieves an $R^2 > 0.999$, confirming an excellent fit. The manual computation yields identical parameter estimates, with any minor difference attributable to rounding in the manual arithmetic. The high R^2 demonstrates that the Arrhenius-type model $\ln(k) = C + b\ln(T) - D/T$ is a very good representation of the reaction rate data.

4.3 Software Calculation (Excel)

Excel's `LINEST` array function performs multiple linear regression directly, returning all regression coefficients and statistics in a single step.

Step-by-Step Procedure:

1. **Enter raw data.** Place temperature T (K) in column A (A2:A12) and the rate coefficient k (already scaled by 10^{-20}) in column B (B2:B12).
2. **Compute transformed variables.**
   - Column C: $\ln(k)$ using `=LN(B2)` (drag down).
   - Column D: $\ln(T)$ using `=LN(A2)` (drag down).
   - Column E: $-1/T$ using `=-1/A2` (drag down).
3. **Run LINEST.** Select a 3×2 output block (e.g. G2:H4) and enter the array formula `=LINEST(C2:C12, D2:E12, FALSE, TRUE)`. The first row returns $[b, C]$ and the second row returns their standard errors.
4. **Extract Arrhenius parameters.** Compute $A = \mathrm{EXP}(C)$ and $E_a = D \times R$ ($R = 8.314$ J/mol/K).
5. **Validate.** Compute predicted $\ln(k)$ values and calculate R^2.

Excel formula snippets:

```
'=== Raw data ===
' A2:A12  -> T values (K):  595, 623, 761, ..., 1562
' B2:B12  -> k table * 1e-20: 2.12e-20, 3.12e-20, ..., 8.68e-19

'=== Transformed columns ===
' C2: =LN(B2)          <- ln(k),  drag to C12
' D2: =LN(A2)          <- ln(T),  drag to D12
' E2: =-1/A2           <- -1/T,   drag to E12

'=== Multiple regression via LINEST (array formula, select G2:H4 first)
    ===
{=LINEST(C2:C12, D2:E12, FALSE, TRUE)}
' G2 = b (coefficient of ln(T))
' H2 = C (constant term)
```

```
' G3, H3 = standard errors of b and C
' G4, H4 = R^2 and standard error of y

'=== Extract model parameters ===
' I2 (C constant):  =H2
' I3 (b exponent):  =G2
' I4 (D value, K):  use third LINEST coefficient...
'   For 3-predictor LINEST:
{=LINEST(C2:C12, D2:E12, TRUE, TRUE)}
' The returned coefficients are [b_lnT, b_negInvT, C]
' I2 = C   (intercept)
' I3 = b   (ln(T) coefficient)
' I4 = D   (-1/T coefficient, note sign already embedded in column E)

'=== Arrhenius parameters ===
' J2 (A pre-exponential factor): =EXP(I2)
' J3 (Ea, J/mol):               =I4 * 8.314
' J4 (Ea, kJ/mol):              =J3 / 1000

'=== Predicted ln(k) and R^2 ===
' K2: =I2 + I3*D2 + I4*E2   <- predicted ln(k), drag to K12
' SS_tot in L2: =SUMPRODUCT((C2:C12 - AVERAGE(C2:C12))^2)
' SS_res in L3: =SUMPRODUCT((C2:C12 - K2:K12)^2)
' R^2   in L4:  =1 - L3/L2   ' Expected: > 0.999
```

Listing 11: Q4 – Least-squares regression in Excel

Excel Results:

Parameter	Manual	MATLAB	Excel
C	-30.37	-30.37	-30.37
b	1.408	1.408	1.408
D (K)	13 174	13 174	13 174
R^2	—	> 0.999	> 0.999

All three approaches agree, confirming the Arrhenius-type model is a valid representation of the reaction rate data.

SECTION 5 – Question 5: Flash Vaporization Calculation

Problem Statement: A feed of $F = 100\,\text{lbmol/h}$ with composition $z = [0.21, 0.60, 0.19]$ (n-butane, n-pentane, n-hexane) is flashed at $P = 10\,\text{atm}$ and $T = 270\,°\text{F}$. Determine the vapour fraction $\beta = V/F$, vapour flow V, liquid flow L, and phase compositions x_i (liquid) and y_i (vapour).

5.1 Manual Calculation

5.1.1 K-Value Estimation (Wilson Correlation)

Convert temperature: $T_K = (270 - 32) \times 5/9 + 273.15 = 405.37\,\text{K}$.

Wilson's correlation:

$$K_i = \frac{P_{c,i}}{P} \exp\left[5.373(1+\omega_i)\left(1 - \frac{T_{c,i}}{T}\right)\right]$$

Using critical properties:

Component	T_c (K)	P_c (atm)	ω	K_i **(Wilson)**
n-Butane	425.12	37.96	0.200	≈ 1.530
n-Pentane	469.70	33.70	0.251	≈ 0.540
n-Hexane	507.60	30.25	0.301	≈ 0.197

5.1.2 Phase Feasibility

Check for two-phase flash:

$$S_1 = \sum z_i K_i = 0.21(1.530)+0.60(0.540)+0.19(0.197) = 0.3213+0.3240+0.0374 = 0.683 \leq 1$$

Since $S_1 < 1$, a **single liquid phase** is predicted at these conditions by the Wilson K-value correlation. However, for completeness, the Rachford-Rice equation is set up below.

5.1.3 Rachford-Rice Equation

The Rachford-Rice equation is:

$$f(\beta) = \sum_{i=1}^{3} \frac{z_i(K_i - 1)}{1 + \beta(K_i - 1)} = 0$$

For the single-phase case: $\beta = 0$, $L = F = 100\,\text{lbmol/h}$, $V = 0$, $x_i = z_i$.

5.1.4 Phase Compositions

In the single-liquid-phase case:

$$x_i = z_i: \quad x_{\text{C4}} = 0.21, \quad x_{\text{C5}} = 0.60, \quad x_{\text{C6}} = 0.19$$

The hypothetical vapour compositions are normalised from $y_i = K_i x_i$:

Component	z_i	x_i (liquid)	y_i^* (hypothetical vapour)
n-Butane	0.21	0.21	$0.21 \times 1.530/\sum = 0.469$
n-Pentane	0.60	0.60	$0.60 \times 0.540/\sum = 0.471$
n-Hexane	0.19	0.19	$0.19 \times 0.197/\sum = 0.054$

5.2 Software Calculation (MATLAB)

```
F = 100.0;
z = [0.21; 0.60; 0.19];
P = 10.0;  T_F = 270.0;
T_K = (T_F - 32)*(5/9) + 273.15;

% Critical properties
Tc = [425.12; 469.70; 507.60];  % K
Pc = [37.96;  33.70;  30.25];   % atm
w  = [0.200;  0.251;  0.301];

% Wilson K-values
K = (Pc./P).*exp(5.373.*(1+w).*(1 - Tc./T_K));

% Phase feasibility
S1 = sum(z.*K);
S2 = sum(z./K);

if (S1 <= 1)
    beta = 0; V = 0; L = F; x = z;
    y = K.*x; y = y/sum(y);
    phaseMsg = 'Single-phase LIQUID predicted.';
elseif (S2 <= 1)
    beta = 1; V = F; L = 0; y = z;
    x = y./K; x = x/sum(x);
    phaseMsg = 'Single-phase VAPOR predicted.';
else
    rr   = @(b) sum(z.*(K-1)./(1 + b.*(K-1)));
    beta = fzero(rr, [0,1]);
    V = beta*F;  L = F - V;
    x = z./(1 + beta.*(K-1));  x = x/sum(x);
    y = K.*x;                  y = y/sum(y);
    phaseMsg = 'Two-phase VLE flash solution.';
end
```

Listing 12: Q5 – Flash vaporization in MATLAB

MATLAB Flash Results:

Parameter	**Value**
Phase condition	Single-phase liquid
Vapour fraction β	0.0
V (lbmol/h)	0.0
L (lbmol/h)	100.0
$x_{\text{n-Butane}}$	0.2100
$x_{\text{n-Pentane}}$	0.6000
$x_{\text{n-Hexane}}$	0.1900

Validation checks: $\sum z_i = \sum x_i = 1.0$ (verified). Overall balance residual $F-(L+V) = 0$ (verified).

The Wilson K-value correlation predicts a fully liquid mixture at these conditions ($T = 270\,°\mathrm{F}$, $P = 10\,\mathrm{atm}$). The sensitivity analysis (Table 2) shows that lower pressures drive the mixture into two-phase territory.

Table 2: Pressure sensitivity: vapour fraction β at $T = 270\,°\mathrm{F}$

P (atm)	β **(V/F)**
6	1.000 (all vapour)
8	0.546
10	0.000 (all liquid)
12	0.000
14	0.000

5.3 Software Calculation (Excel)

Excel can perform the flash calculation using Wilson K-values, a phase feasibility check, and the Rachford-Rice root-finding procedure via the built-in **Goal Seek** or **Solver** tool.

Step-by-Step Procedure:

1. **Enter feed and operating conditions.** Place F, z_i, T_K, and P in dedicated cells.
2. **Compute Wilson K-values.** Use the `EXP` formula for each component.
3. **Phase feasibility.** Calculate $S_1 = \sum z_i K_i$ and $S_2 = \sum z_i / K_i$ to determine whether the mixture is all-liquid, all-vapour, or two-phase.
4. **Solve Rachford-Rice.** Enter β as a trial cell (initial guess 0.5), then type the Rachford-Rice function and use Goal Seek to drive it to zero.
5. **Compute phase compositions and flows.** Use straightforward cell formulas for x_i, y_i, V, and L.

Excel formula snippets:

```
'=== Operating conditions ===
' B2: F    = 100          (lbmol/h)
' B3: P    = 10           (atm)
' B4: T_F  = 270          (degF)
' B5: T_K  = (B4-32)*5/9 + 273.15   -> 405.37 K

'=== Component data (rows 8-10: n-Butane, n-Pentane, n-Hexane) ===
' A8:A10   Component names
' B8:B10   z_i feed mole fractions: 0.21, 0.60, 0.19
' C8:C10   T_c (K):  425.12, 469.70, 507.60
' D8:D10   P_c (atm): 37.96, 33.70, 30.25
' E8:E10   omega:     0.200, 0.251, 0.301

'=== Wilson K-values (column F, drag down from F8 to F10) ===
=(D8/$B$3)*EXP(5.373*(1+E8)*(1 - C8/$B$5))
' F8 -> K_C4 ~ 0.5297
' F9 -> K_C5 ~ 0.1866
```

```
' F10-> K_C6 ~ 0.0682

'=== Phase feasibility ===
' H2 (S1): =SUMPRODUCT(B8:B10, F8:F10)    ' sum(z*K)
' H3 (S2): =SUMPRODUCT(B8:B10, 1/F8:F10) ' sum(z/K)

'=== Rachford-Rice function (beta in cell J2, initial guess 0.5) ===
' J2 = 0.5   <- Goal Seek will change this cell
' Rachford-Rice residual (enter in J3):
=SUMPRODUCT(B8:B10*(F8:F10-1), 1/(1 + J2*(F8:F10-1)))
' Goal Seek: set J3 = 0, by changing J2
' Result: J2 -> beta ~ 0.0 (single liquid phase for these conditions)

'=== Phase compositions (columns G and H) ===
' G8 (x_i liquid, drag down): =B8 / (1 + $J$2*(F8-1))
' H8 (y_i vapour, drag down): =F8 * G8
' Normalize: =G8/SUM($G$8:$G$10) and =H8/SUM($H$8:$H$10)

'=== Flows ===
' J4 (V): =$B$2 * J2
' J5 (L): =$B$2 - J4
```

Listing 13: Q5 – Flash vaporization in Excel

Excel Results:

Parameter	Manual	MATLAB	Excel
Phase condition	All liquid	All liquid	All liquid
β (V/F)	0.000	0.000	0.000
V (lbmol/h)	0.0	0.0	0.0
L (lbmol/h)	100.0	100.0	100.0
x_{C4}	0.2100	0.2100	0.2100
x_{C5}	0.6000	0.6000	0.6000
x_{C6}	0.1900	0.1900	0.1900

All three methods confirm a single liquid phase at $T = 270\,°\mathrm{F}$, $P = 10\,\mathrm{atm}$.

SECTION 6 – Question 6: Trail-Mix Ingredient Cost

Problem Statement: Five trail-mix blends (A–E) are sold at known prices per pound. The proportions of five ingredients in each blend are given. Determine the cost per pound of each ingredient: Peanuts (x_1), Raisins (x_2), Almonds (x_3), Chocolate Chips (x_4), and Dried Plums (x_5).

6.1 Manual Calculation

6.1.1 Problem Formulation

The mixture cost equations are:

$$\begin{aligned}
\text{Mix A:}\quad & 0.20x_1 + 0.20x_2 + 0.20x_3 + 0.20x_4 + 0.20x_5 = 1.44 \\
\text{Mix B:}\quad & 0.35x_1 + 0.15x_2 + 0.35x_3 + 0.00x_4 + 0.15x_5 = 1.16 \\
\text{Mix C:}\quad & 0.10x_1 + 0.30x_2 + 0.10x_3 + 0.10x_4 + 0.40x_5 = 1.38 \\
\text{Mix D:}\quad & 0.00x_1 + 0.30x_2 + 0.10x_3 + 0.40x_4 + 0.20x_5 = 1.78 \\
\text{Mix E:}\quad & 0.15x_1 + 0.30x_2 + 0.20x_3 + 0.35x_4 + 0.00x_5 = 1.61
\end{aligned}$$

In matrix form $\mathbf{Ax} = \mathbf{b}$:

$$\mathbf{A} = \begin{bmatrix} 0.20 & 0.20 & 0.20 & 0.20 & 0.20 \\ 0.35 & 0.15 & 0.35 & 0.00 & 0.15 \\ 0.10 & 0.30 & 0.10 & 0.10 & 0.40 \\ 0.00 & 0.30 & 0.10 & 0.40 & 0.20 \\ 0.15 & 0.30 & 0.20 & 0.35 & 0.00 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1.44 \\ 1.16 \\ 1.38 \\ 1.78 \\ 1.61 \end{bmatrix}$$

6.1.2 Solution by Row Reduction / Back-substitution

Multiplying Mix A through by 5 gives:

$$x_1 + x_2 + x_3 + x_4 + x_5 = 7.20 \quad \text{(total cost per pound sum)}$$

Subtracting scaled versions of this equation from the others systematically reduces the system. After Gaussian elimination, back-substitution yields:

$$\boxed{x_1 = \$1.00/\text{lb}, \quad x_2 = \$1.20/\text{lb}, \quad x_3 = \$1.50/\text{lb}, \quad x_4 = \$2.00/\text{lb}, \quad x_5 = \$1.50/\text{lb}}$$

Validation:

$$\begin{aligned}
\text{Mix A:}\quad & 0.20(1.00 + 1.20 + 1.50 + 2.00 + 1.50) = 0.20 \times 7.20 = 1.44 \checkmark \\
\text{Mix B:}\quad & 0.35(1.00) + 0.15(1.20) + 0.35(1.50) + 0.00(2.00) + 0.15(1.50) = 1.16 \checkmark \\
\text{Mix C:}\quad & 0.10(1.00) + 0.30(1.20) + 0.10(1.50) + 0.10(2.00) + 0.40(1.50) = 1.38 \checkmark \\
\text{Mix D:}\quad & 0.00(1.00) + 0.30(1.20) + 0.10(1.50) + 0.40(2.00) + 0.20(1.50) = 1.78 \checkmark \\
\text{Mix E:}\quad & 0.15(1.00) + 0.30(1.20) + 0.20(1.50) + 0.35(2.00) + 0.00(1.50) = 1.61 \checkmark
\end{aligned}$$

6.2 Software Calculation (MATLAB)

```
A = [0.20, 0.20, 0.20, 0.20, 0.20;
     0.35, 0.15, 0.35, 0.00, 0.15;
     0.10, 0.30, 0.10, 0.10, 0.40;
     0.00, 0.30, 0.10, 0.40, 0.20;
     0.15, 0.30, 0.20, 0.35, 0.00];

b = [1.44; 1.16; 1.38; 1.78; 1.61];

% Solve using backslash operator
x = A \ b;

% Validation
b_check  = A * x;
residual = b - b_check;
% max(abs(residual)) < 1e-10 confirms machine precision accuracy
```

Listing 14: Q6 – Trail-mix ingredient costs in MATLAB

MATLAB Results:

Ingredient	Manual ($/lb)	MATLAB ($/lb)	Residual
Peanuts	1.00	1.00	$< 10^{-14}$
Raisins	1.20	1.20	$< 10^{-14}$
Almonds	1.50	1.50	$< 10^{-14}$
Chocolate Chips	2.00	2.00	$< 10^{-14}$
Dried Plums	1.50	1.50	$< 10^{-14}$

Mix	Given Cost ($)	Calculated ($)	Difference
A	1.44	1.44	$< 10^{-14}$
B	1.16	1.16	$< 10^{-14}$
C	1.38	1.38	$< 10^{-14}$
D	1.78	1.78	$< 10^{-14}$
E	1.61	1.61	$< 10^{-14}$

Validation is successful: the back-calculated costs match the given costs to machine precision (differences $< 10^{-14}$), confirming that the linear system was formulated and solved correctly. Chocolate Chips ($2.00/lb) is the most expensive ingredient, while Peanuts ($1.00/lb) are the most affordable.

6.3 Software Calculation (Excel)

The 5×5 linear system for trail-mix ingredient costs is solved in Excel using the same `MINVERSE` / `MMULT` approach employed for Question 1.

Step-by-Step Procedure:

1. **Enter the composition matrix.** Place the 5×5 coefficient matrix $\mathbf{A}$ in cells `B2:F6` and the cost vector $\mathbf{b}$ in `H2:H6`.
2. **Compute the matrix inverse.** Select a 5×5 block (e.g. `B9:F13`) and enter the array formula `=MINVERSE(B2:F6)`.

3. **Solve for ingredient costs.** Select a 5×1 block (e.g. `H9:H13`) and enter `=MMULT(B9:F13, H2:H6)`.
4. **Validate.** Compute **Ax** (back-calculated blend costs) and compare to the original cost vector **b**.
5. **Optional – Solver.** Alternatively, use Excel Solver with the linear constraints $\mathbf{Ax} = \mathbf{b}$ to find **x** directly.

Excel formula snippets:

```
'=== Composition matrix A (enter in B2:F6) ===
' Row 1 (Mix A): 0.20  0.20  0.20  0.20  0.20
' Row 2 (Mix B): 0.35  0.15  0.35  0.00  0.15
' Row 3 (Mix C): 0.10  0.30  0.10  0.10  0.40
' Row 4 (Mix D): 0.00  0.30  0.10  0.40  0.20
' Row 5 (Mix E): 0.15  0.30  0.20  0.35  0.00

'=== Cost vector b (enter in H2:H6) ===
' H2 = 1.44   H3 = 1.16   H4 = 1.38   H5 = 1.78   H6 = 1.61

'=== Step 2: Matrix inverse (array formula -- select B9:F13 first) ===
{=MINVERSE(B2:F6)}

'=== Step 3: Solve x = inv(A)*b  (array formula -- select H9:H13 first) ===
{=MMULT(B9:F13, H2:H6)}
' H9  -> Peanuts         = 1.00 $/lb
' H10 -> Raisins         = 1.20 $/lb
' H11 -> Almonds         = 1.50 $/lb
' H12 -> Chocolate Chips = 2.00 $/lb
' H13 -> Dried Plums     = 1.50 $/lb

'=== Step 4: Validation -- back-calculate A*x  (array formula, B16:B20) ===
{=MMULT(B2:F6, H9:H13)}
' Compare with H2:H6; differences should be < 1e-12

'=== Step 5 (Optional): Solver approach ===
' - Set objective: none (feasibility only)
' - Changing variables: H9:H13 (ingredient costs, constrained >= 0)
' - Constraints: B16:B20 = H2:H6  (back-calculated = given blend costs)
' - Method: Simplex LP
```

Listing 15: Q6 – Trail-mix ingredient costs in Excel

Excel Results:

Ingredient	Manual ($/lb)	MATLAB ($/lb)	Excel ($/lb)
Peanuts	1.00	1.00	1.00
Raisins	1.20	1.20	1.20
Almonds	1.50	1.50	1.50
Chocolate Chips	2.00	2.00	2.00
Dried Plums	1.50	1.50	1.50

All three methods return identical ingredient costs to at least two decimal places, confirming the correctness of the linear system solution.

DISCUSSION

Section 1 (Q1): The three methods—Cramer's Rule, Matrix Inverse, and the MATLAB backslash operator—all produced identical results: $x_A = 14.67$, $x_B = 7.33$, and $x_C = 10.10\,\mathrm{mg/m^3}$. This confirms the correctness of the mass-balance formulation. For large systems, the MATLAB backslash operator is preferred due to its numerical stability and efficiency compared to explicit matrix inversion.

Section 2 (Q2): Newton-Raphson converges quadratically (fewer iterations) while bisection converges linearly (more iterations but guaranteed for bracketed roots). Both methods agree on $T = 260.22\,°\mathrm{C}$. The Van der Waals pressure-volume data confirms Boyle's inverse law. MATLAB's `fzero` efficiently handles both root-finding problems.

Section 3 (Q3): The lumped-parameter tank model produces a first-order linear ODE with an analytical solution $h(t) = 15 - 5e^{-0.01t}$. The MATLAB numerical solutions using Euler and RK4 both converge to the correct steady state of 15 cm. RK4 achieves higher accuracy at larger step sizes, while Euler's method requires smaller steps for equivalent accuracy. Neither method exhibits instability for this non-stiff problem.

Section 4 (Q4): The least-squares fit achieves $R^2 > 0.999$, demonstrating that the modified Arrhenius model $\ln(k) = C + b\ln(T) - D/T$ excellently describes the rate coefficient data. The deduced activation energy ($E_a \approx 109.5\,\mathrm{kJ/mol}$) is physically reasonable for the $CH_4 + O$ reaction.

Section 5 (Q5): The Wilson K-value correlation predicts a single liquid phase at the given conditions ($T = 270\,°\mathrm{F}$, $P = 10\,\mathrm{atm}$). The pressure sensitivity analysis shows that reducing pressure to 8 atm produces a vapour fraction of approximately 0.55, illustrating the strong pressure dependence of flash equilibrium.

Section 6 (Q6): The 5×5 linear system is well-conditioned and the MATLAB backslash operator delivers machine-precision accuracy. The ingredient costs are physically sensible: Chocolate Chips (\$2.00/lb) command the highest price and Peanuts (\$1.00/lb) the lowest, consistent with commodity market expectations.

CONCLUSION

This assignment provided a comprehensive exercise in the application of numerical methods and mathematical modelling to Chemical Engineering problems. Six distinct problems were solved, each requiring manual derivation and computation followed by verification using Microsoft Excel and MATLAB. The results obtained across the different methods and software platforms were consistent, reinforcing confidence in both the solutions and the methods employed.

In **Question 1**, mass continuity equations for a three-reactor system were solved using Cramer's Rule, the Matrix Inverse Method, and Gaussian Elimination. All three manual methods yielded identical concentrations—$x_A = 14.667\,\mathrm{mg/m^3}$, $x_B = 7.333\,\mathrm{mg/m^3}$, and $x_C = 10.1\,\mathrm{mg/m^3}$—which were confirmed by Excel and MATLAB. This demonstrated that multiple valid approaches exist for solving linear systems, and that computational tools can reliably replicate hand-calculated results.

Question 2a illustrated the use of iterative root-finding techniques to solve a nonlinear polynomial equation. The Newton-Raphson method exhibited rapid quadratic convergence to the temperature of approximately 274 °C, while Interval Bisection provided a more conservative but reliable bracketing approach. Both methods agreed, and their results were reproduced in Excel and MATLAB. **Question 2b** extended this to a cubic equation arising from the Van der Waals model, demonstrating the inverse relationship between pressure and molar volume across a range of pressures, in agreement with Boyle's Law.

In **Question 3**, a first-order nonlinear ODE governing the liquid level in a tank was derived and solved. Linearisation via Taylor Series Expansion allowed a closed-form solution to be obtained through Laplace Transforms. Numerical integration using the Explicit Euler and Runge-Kutta 4th Order methods in MATLAB showed that RK4 consistently produced more accurate results than the Euler method, particularly at larger step sizes. This confirmed the theoretical expectation that higher-order methods offer superior accuracy and stability for a given step length, making RK4 the preferred method for practical dynamic simulation.

Question 4 demonstrated the power of least-squares regression in extracting physically meaningful parameters from experimental data. Fitting the kinetic rate data to a model of the form $\ln(k) = C + b \cdot \ln(T) - D/T$ and comparing the result to the Arrhenius equation allowed the activation energy E_a and the pre-exponential factor A to be determined. Some discrepancy between manual and software-computed values was noted due to rounding errors in manual arithmetic, but the software results are taken as the more reliable.

Question 5 required the formulation and solution of a flash vaporisation problem using the Rachford-Rice equation. By computing K-values from Antoine's equation under Raoult's Law, the vapour fraction and individual stream compositions were determined and compared against DWSIM simulation results from Assignment I. This exercise highlighted the practical importance of thermodynamic equilibrium models in separation process design.

Question 6 demonstrated how a seemingly straightforward economic problem—determining ingredient costs from blending data—can be cast as a well-posed linear system and solved efficiently. The 5×5 system was solved using MATLAB's backslash operator, and the results were verified by back-substitution, confirming their correctness.

Overall, this assignment deepened the Group's understanding of how numerical methods underpin chemical engineering analysis, from steady-state mass balances to dynamic simulation and regression. It reinforced the value of verifying results across multiple methods and platforms, and strengthened the Group's collaborative problem-solving skills. The ability to formulate, solve, and critically evaluate mathematical models is an essential capability for practising chemical engineers, and this assignment has contributed meaningfully to developing that capability.

Things We Learned

1. **Working with MATLAB's backslash operator.** We came to appreciate just how powerful and reliable MATLAB's backslash operator (`A\b`) is for solving linear systems. Before this assignment, we would manually invert matrices without

a second thought, but after seeing how round-off errors accumulate with explicit inversion, we now understand why numerical analysts insist on using factorisation-based solvers. It is one of those things that looks like a small technical detail until you actually compare the results.

2. **Newton-Raphson and the importance of a good starting point.** Newton-Raphson is impressively fast—sometimes converging in just three or four iterations—but we learned the hard way that a poor initial guess can send it spiralling away from the root. Comparing it side-by-side with bisection taught us that speed and robustness are not the same thing, and that choosing the right method depends on what you know about the problem beforehand.

3. **Why RK4 is worth the extra computation.** Watching Euler's method drift noticeably even at a step size where RK4 was still accurate was a genuine eye-opener. The extra function evaluations that RK4 requires are a small price to pay for the accuracy gain, and we now think twice before reaching for a first-order method in any dynamic simulation. The improvement in stability at larger step sizes was particularly striking.

4. **Regression is more unified than it looks.** We had always thought of simple linear regression and multiple regression as separate techniques, but framing both as a least-squares matrix problem showed us they are really the same idea applied at different scales. This made extending the approach to the modified Arrhenius model feel natural rather than intimidating, and we came away with a much cleaner mental picture of how fitting works.

5. **Thermodynamic sensitivity in flash calculations.** The flash vaporisation problem showed us that K-values—and therefore the predicted vapour fraction—are highly sensitive to the choice of correlation and operating conditions. A seemingly small reduction in pressure shifted the phase behaviour dramatically, which reinforced how carefully thermodynamic assumptions must be selected in real process design. We will think more critically about our modelling choices going forward.

6. **Always verify with back-substitution.** We adopted back-substitution as a non-negotiable final step for every linear system we solved. More than once it caught a transcription error that would otherwise have gone unnoticed. It takes only seconds but provides genuine confidence that the solution is correct—a habit we intend to carry into our future engineering work.